

Project 3 Part 2

Hyeonseo, Sunho, Oghap

March 2026

Part 2A: Misconfiguration

(i) Accepting Self-Signed Certificates

In `client_pt1.py`, the client currently verifies the server certificate using:

```
context = ssl.create_default_context()
context.load_verify_locations("server.crt")
```

A misconfiguration would be changing it to something like:

```
context = ssl.create_default_context()
context.verify_mode = ssl.CERT_NONE
```

If this happened, the client would stop checking whether the server certificate is trusted. That means it could accept a fake or self-signed certificate. In that case, an attacker could pretend to be the server and intercept the connection.

This would be security vulnerability because TLS provides two guarantees to provide confidentiality and authentication. Disabling certificate verification breaks the authentication guarantee. An attacker who can intercept network traffic could present a fake certificate and the client would accept it, establishing an encrypted channel with the attacker rather than the real server. The data would still be encrypted, but the attacker on the other end would be able to read it all. Weak cipher suites weakens the confidentiality of the connection even when everything is configured correctly.

(ii) Permitting Weak Cipher Suites

In `server_pt1.py`, the server currently creates a TLS context and loads its certificate normally. A weak configuration could look like this:

```
context.set_ciphers("RC4")
```

This would allow an old and weak cipher suite. If weak ciphers are allowed, then the TLS connection is not as secure and attackers may have a better chance of breaking the encryption or weakening the session.

Part 2B: Mallory-in-the-Middle

If certificate verification is turned off in the client, then a man-in-the-middle attack becomes possible. For example, an attacker could stand between the client and server and give the client a fake certificate. Since the client is not checking certificates anymore, it would accept the attacker's certificate and continue the connection.

Then the attacker could read the client's request, forward it to the real server, get the server's response, and send it back to the client. The client may think the connection is secure, but the attacker is actually in the middle.

Proper certificate verification stops this attack. In our code, the client loads the trusted certificate using `context.load_verify_locations("server.crt")`. Because of that, if an attacker shows a fake certificate, the handshake should fail instead of continuing.